带表头结点的单链表

- **表头结点位于表的最前端，本身不带数据，仅标志表头。**
- **设置表头结点的目的：**

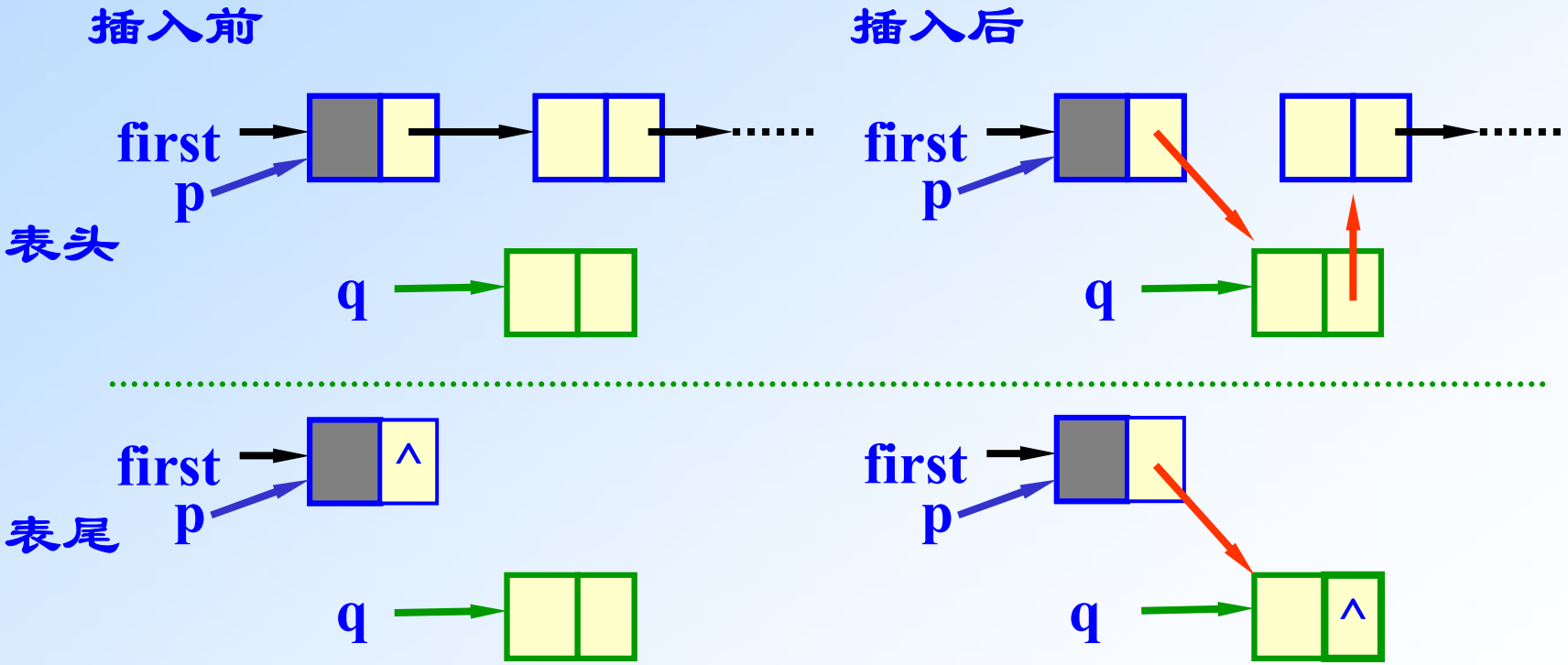
  简化链表操作的实现。

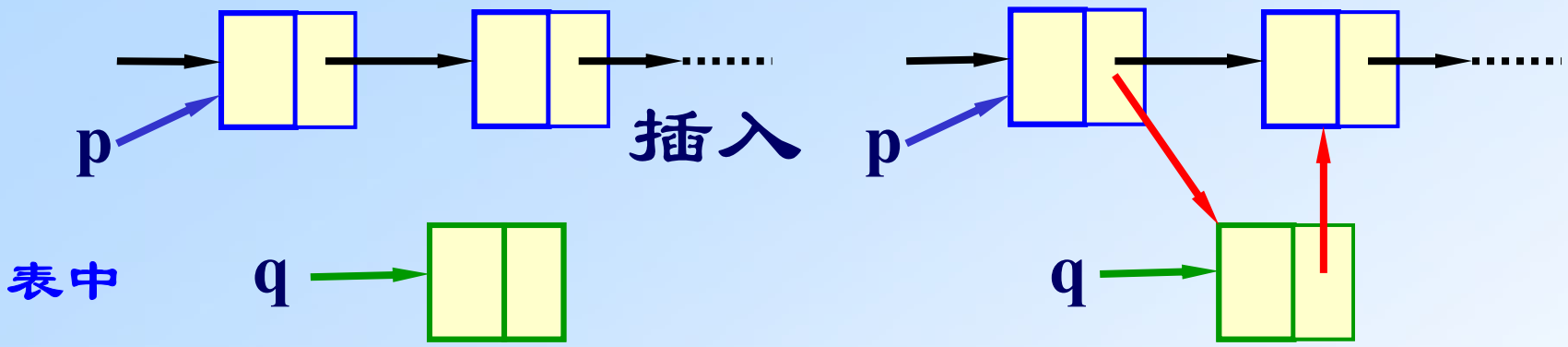


非空表　　空表

▪插入

```
q->link = p->link;
p->link = q;
```



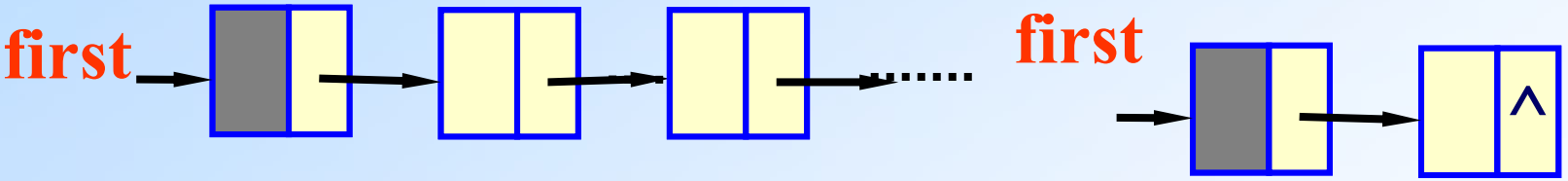

p
插入
表中
q
p
q

```
int Insert (LinkList first, ListData x, int i ) {
//将新元素 x 插入在链表中第 i 号结点位置
    ListNode * p = Locate ( first, i-1 );
    if ( p == NULL ) return 0;     //参数i值不合理返回0
    ListNode * newnode =           //创建新结点
       (ListNode *) malloc (sizeof (ListNode) );
    newnode->data = x;
    newnode->link = p->link;       //链入
    p->link = newnode;
    return 1;
}
```

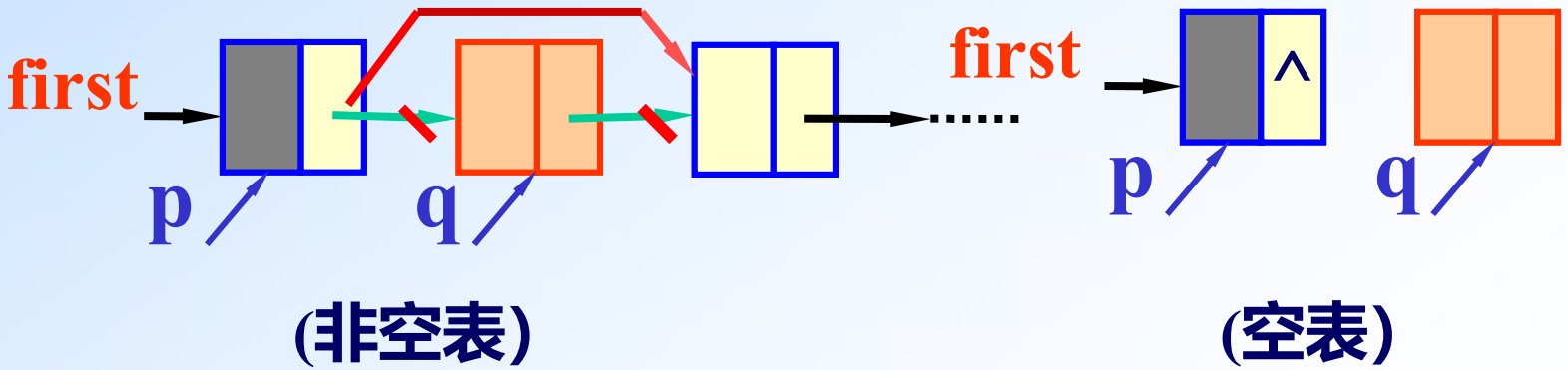
▪删除

```
q = p->link;
p->link = q->link;
free(q);
```

删除前



删除后

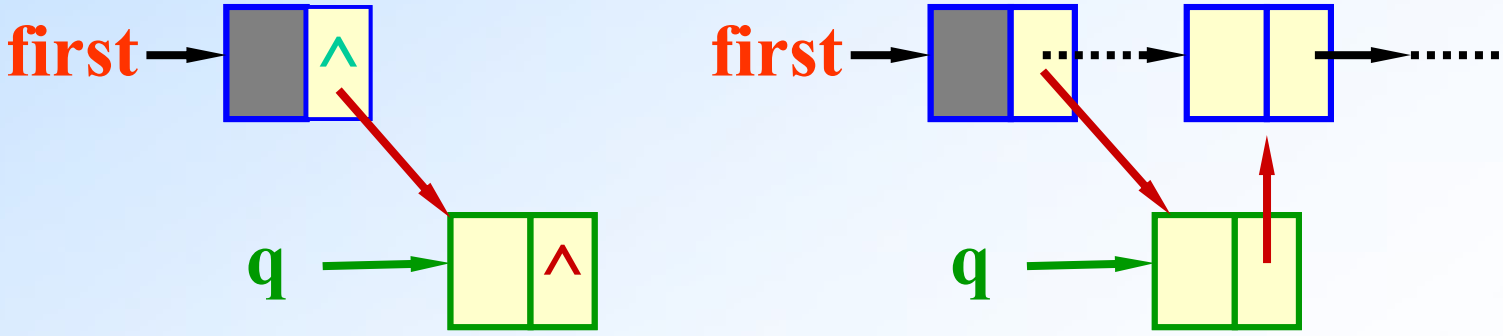


(非空表)　　(空表)

```
int delete ( LinkList first, int i ) {
    //将链表第 i 号元素删去
    ListNode * p, * q
    p = Locate ( first, i-1 ); //寻找第i-1个结点
    if ( p == NULL || p->link == NULL)
        return 0;//i值不合理或空表
    q = p->link;
    p->link = q->link;          //删除结点
    free ( q );                 //释放
    return 1;
}
```

前插法建立单链表

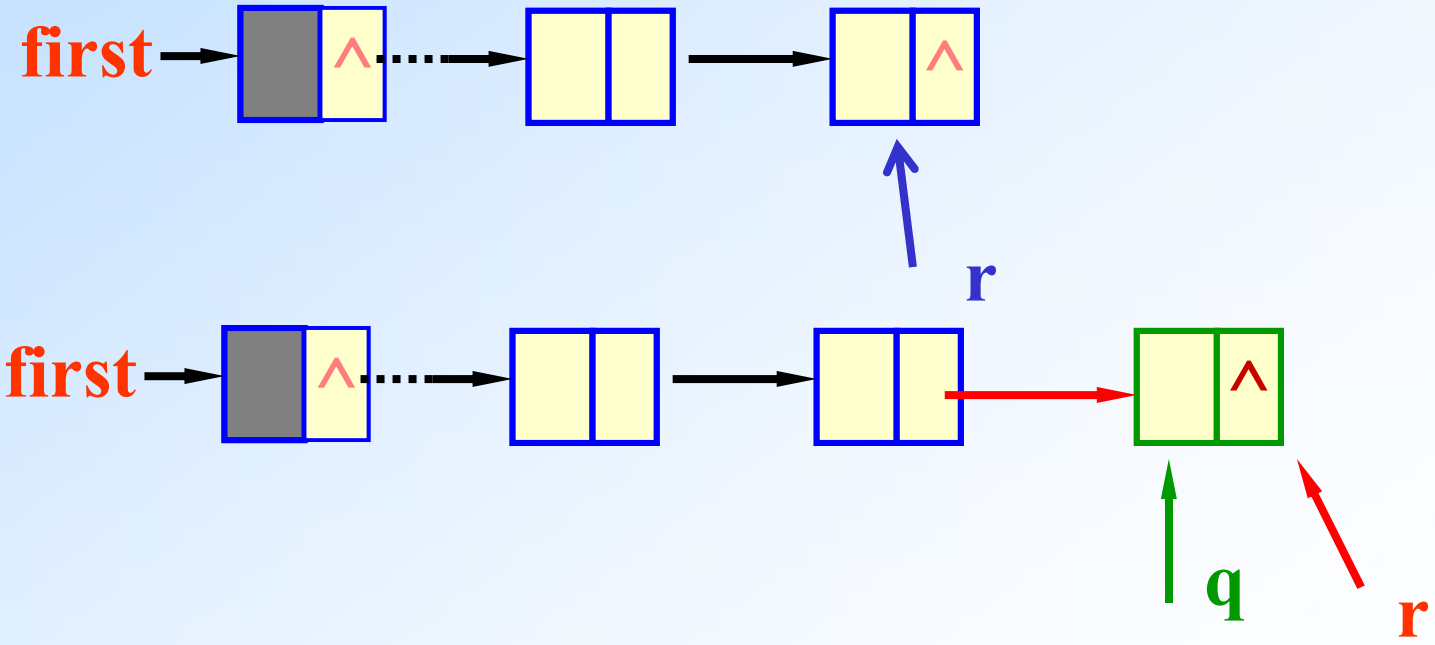
- 从一个空表开始，重复读入数据：
  - 生成新结点
  - 将读入数据存放到新结点的数据域中
  - 将该新结点插入到链表的前端
- 直到读入结束符为止。



```
LinkList createListF ( void ) {
    char ch;   ListNode *q;
    LinkList  head =                    //建立表头结点
       (LinkList) malloc (sizeof (ListNode));
    head->link = NULL;
    while ( (ch = getchar( ) ) != '\n' ) {
        q = (ListNode *) malloc (sizeof(ListNode));
        q->data = ch;                   //建立新结点
        q->link = head->link;  //插入到表前端
        head->link = q;
    }
    return head;
}
```

后插法建立单链表

- 每次将新结点加在链表的表尾；
- 尾指针r ,总是指向表中最后一个结点，新结点插在它的后面；



```
LinkList createListR ( void ) {
    char ch;
    LinkList  head =        //建立表头结点
       (LinkList) malloc (sizeof (ListNode));
    ListNode *q, *r = head;
    while ( (ch = getchar( ) ) != '\n' ) {
       q = (ListNode *) malloc (sizeof(ListNode));
        q->data = ch;            //建立新结点
        r ->link = q;   r =q;    //插入到表末端
    }
    r ->link = NULL;    //尾结点link置空
    return head;
}
```

单链表清空

```
void makeEmpty ( LinkList first ) {
//删去链表中除表头结点外的所有其它结点
    ListNode *q;
    while ( first->link != NULL ) {//当链不空时，循环逐个删去所有结点
        q = first->link;  first->link = q->link;
        free( q );        //释放
    }
    last=first;
}
```

计算单链表长度

```
int Length ( LinkList first ) {
   ListNode *p = first->link; //指针 p 指示第一个结点
    int count = 0;
    while ( p != NULL ) {      //逐个结点检测
         p = p->link;  count++;
     }
     return count;
}
```

按值查找

```
ListNode * Find ( LinkList first, ListData value )
   {
        //在链表中从头搜索其数据值为value的结点
    ListNode * p = first->link;
        //指针 p 指示第一个结点
    while ( p != NULL && p->data != value )
       p = p->link;
    return p;
}
```

按序号查找（定位）

```
ListNode * Locate ( LinkList first, int i ) {
//返回表中第 i 个元素的地址
    if ( i < 0 ) return NULL;
    ListNode * p = first;   int k = 0;
    while ( p != NULL && k < i )
       { p = p->link;  k++; }     //找第 i 个结点
    if  ( k == i ) return p; //返回第 i 个结点地址
    else return NULL;
}
```

随堂习题

- **L为带表头节点的链表，P不为首元素，请排序以下代码删除P的直接前驱**

```
(a) P->next = P->next->next;
(b) While(P->next->next!=Q) P=P->next;
(c) Q=P;
(d) free(Q);
(e) Q=P->next;
(f) P=L;
```

答案：(c)(f)(b)(e)(a)(d)

静态链表

用一维数组描述线性链表

0		1
1	ZHANG	2
2	WANG	3
3	LI	4
4	ZHAO	5
5	WU	-1
6		

修改前

0		1
1	ZHANG	2
2	WANG	6
3	LI	5
4	ZHAO	5
5	WU	-1
6	CHEN	3

(在wang和li之间插入chen, 删除zhao) 修改后

- 定义

```
const int MaxSize = 100;      //静态链表大小

typedef  int ListData;

typedef struct node {         //静态链表结点
    ListData data;
    int link;
} SNode;

typedef struct {              //静态链表
    SNode Nodes[MaxSize];
    int newptr;               //当前可分配空间首地址（标号）
} SLinkList;
```

链表空间初始化

```
void InitList ( SLinkList* SL ) {
    SL->Nodes[0].link = 1;
    SL->newptr = 1;   //当前可分配空间从 1 开始
    //建立带表头结点的空链表
    for ( int i = 1; i < MaxSize-1; i++ )
        SL->Nodes[i].link = i+1; //构成空闲链接表
    SL->Nodes[MaxSize-1].link = -1;
        //链表收尾
}
```

在静态链表中查找具有给定值的结点

```
int Find ( SLinkList SL, ListData x ) {
    int p = SL.Nodes[0].link; //指针 p 指向链表第一个结点
        while ( p != -1 )//逐个查找有给定值的结点
        if ( SL.Nodes[p].data != x)
            p = SL.Nodes[p].link;
        else break;
        return p;
}
```

在静态链表中查找第 i 个结点

```
int Locate ( SLinkList SL, int i ) {
    if ( i < 0 ) return -1;//参数不合理
    if ( i == 0 ) return 0;
    int j = 0, p = SL.Nodes[0].link;
    while ( p != -1 && j < i ) {//循环查找第 i 号结点
        p = SL.Nodes[p].link;
        j++;
    }
    return p;
}
```

在静态链表第 i 个结点处插入一个新结点

```
int Insert ( SLinkList* SL, int i, ListData x ) {
    int p = Locate ( SL, i-1 );
    if ( p == -1 ) return 0;          //找不到结点
    int q = SL->newptr;               //分配结点
    SL->newptr = SL->Nodes[SL->newptr].link;
    SL->Nodes[q].data = x;
    SL->Nodes[q].link = SL.Nodes[p].link;
    SL->Nodes[p].link = q;            //插入
    return 1;
}
```

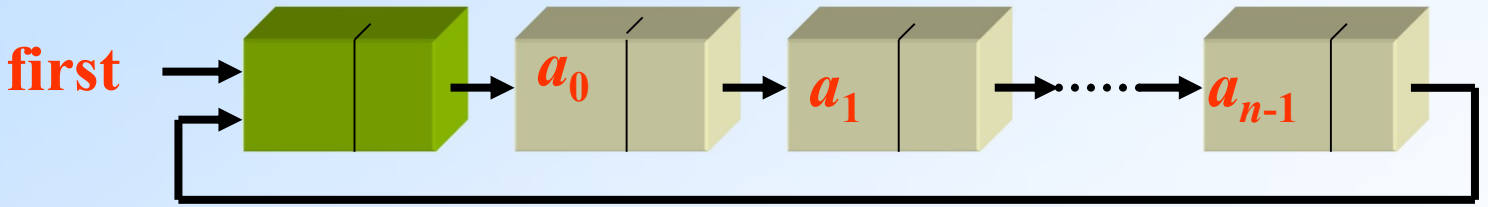
在静态链表中释放第 i 个结点

```
int Remove ( SLinkList* SL, int i ) {
    int p = Locate ( SL, i-1 );
    if ( p == -1 ) return 0;        //找不到结点
    int q = SL->Nodes[p].link;      //第 i 号结点
    SL->Nodes[p].link = SL->Nodes[q].link;
    SL->Nodes[q].link = SL->newptr;     //释放
    SL->newptr = q;
    return 1;
}
```

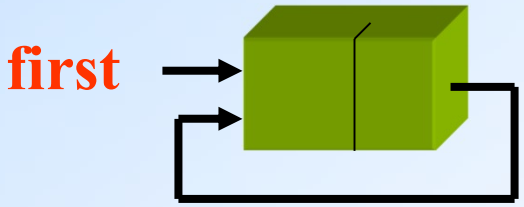
循环链表 (Circular List)

- **特点:**最后一个结点的 link 指针不为NULL，而是指向头结点。只要已知表中某一结点的地址，就可搜寻所有结点的地址。
- **存储结构:**链式存储结构

  带表头结点的循环链表

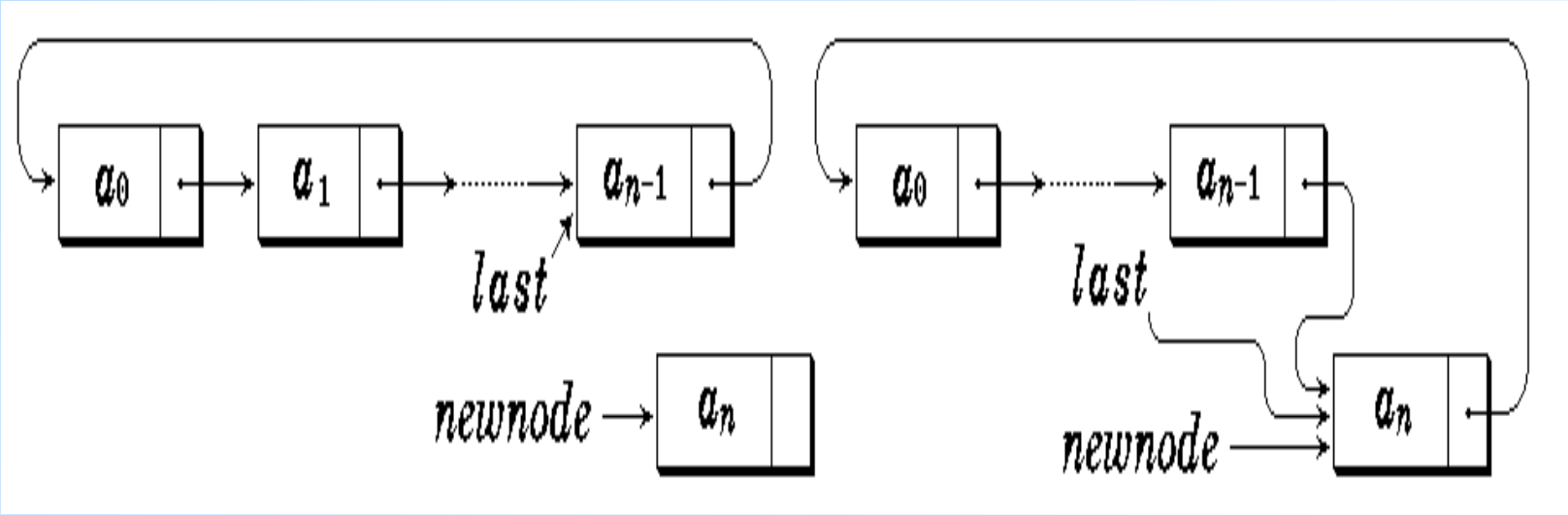


(非空表)



(空表)

循环链表的插入



循环链表类型定义

- #define datatype int
- typedef struct node{

    datatype data; /*数据元素类型*/

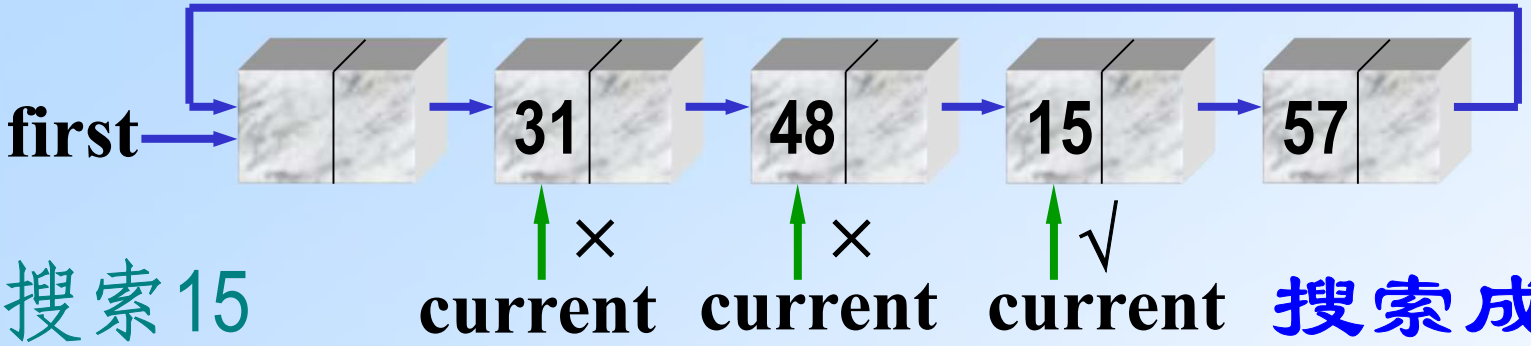
    stuct node *next; /*指针类型*/

  }node, *clklist /*结点类型*/

  其中node是节点类型

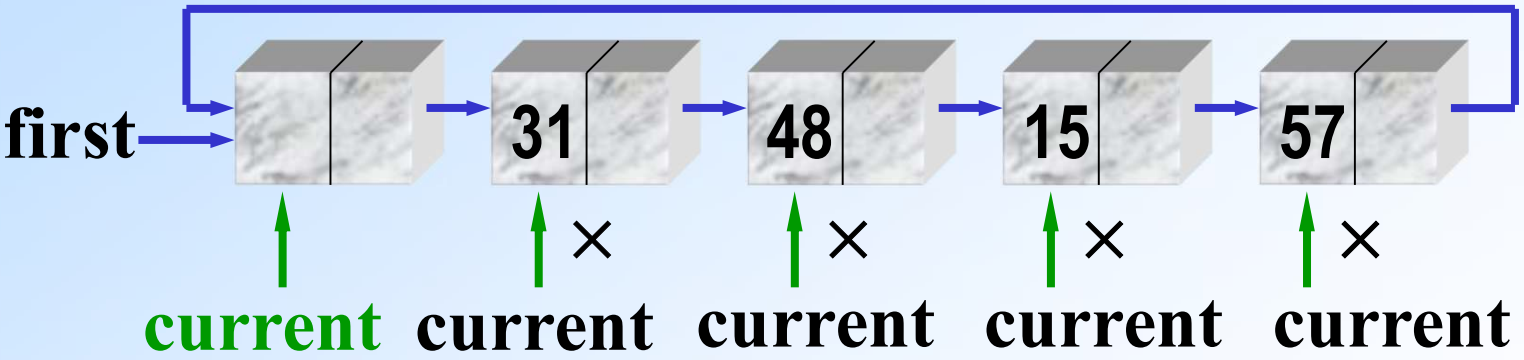
  clklist是指向node类型的指针

循环链表的搜索算法



搜索15

搜索成功



搜索25

搜索不成功

循环链表的按号搜索算法（C描述）

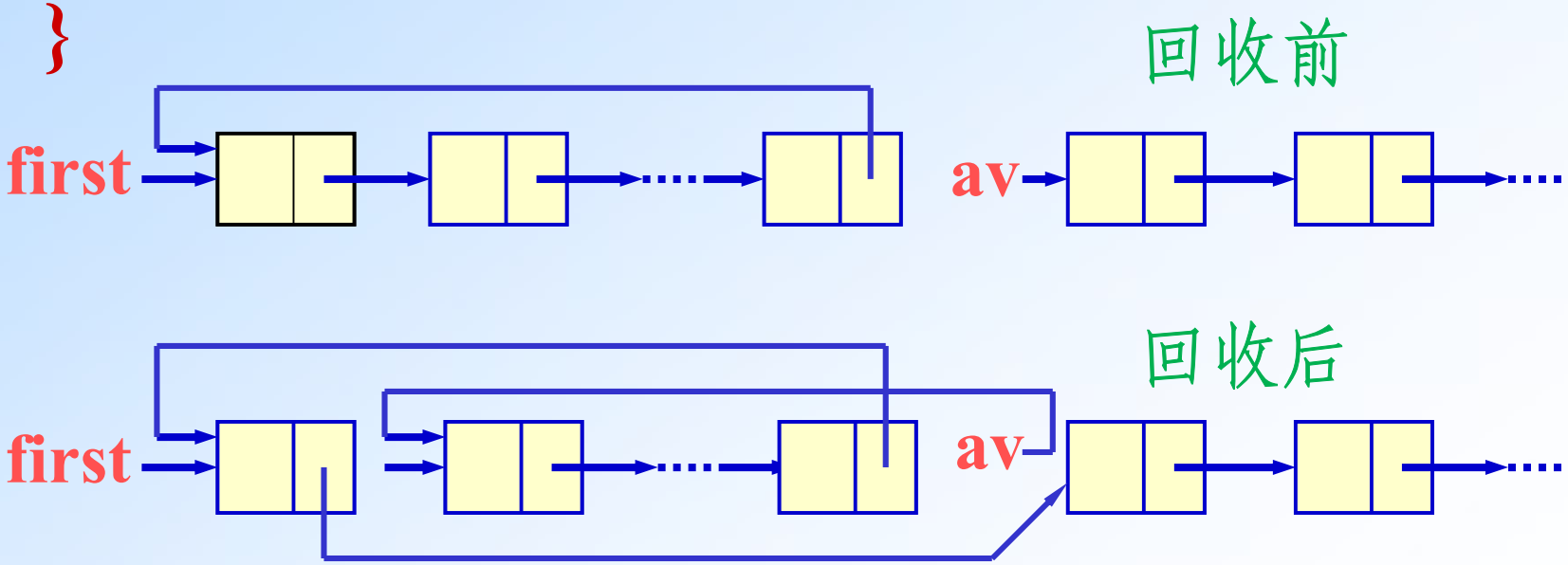
```
node *find_clklist(clklist l ,int i){
        clklist p = l;
        int j = 0; /*初始化*/
        while((p->next!=l)&&(j<i)){
        /*p所指不是尾结点且还没找到时*/
                p=p->next; /*找下一个*/
                j++; /*计数器加1*/
        }
        if(j == i) return(p); /*找到第i个*/
        else return(NULL);
}
```

循环链表的按值搜索算法（C++描述）

```
template <class Type>
CircListNode<Type> * CircList<Type> ::
Find ( Type value ) {
//在链表中从头搜索其数据值为value的结点
    CircListNode<Type> * p = first->link;
    //检测指针 p 指示第一个结点
    while ( p != first && p->data != value )
        p = p->link;
    return p;
}
```

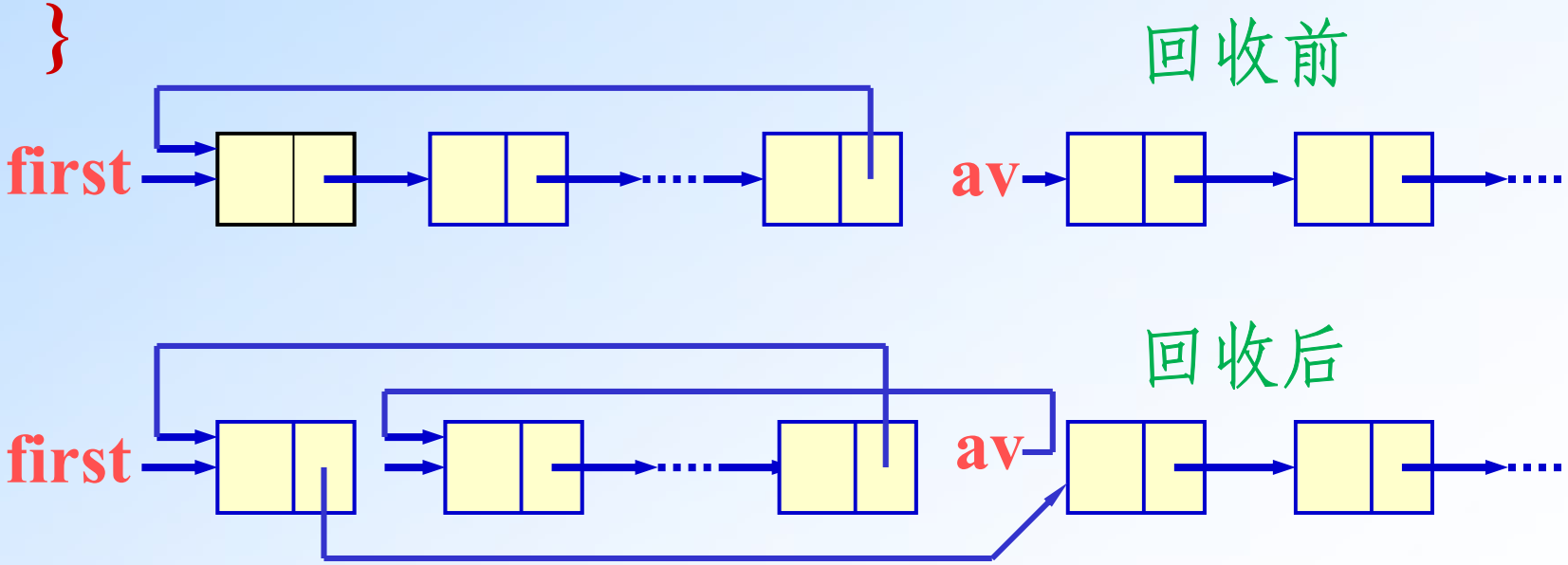
利用可利用空间表回收循环链表（C描述）

```
if ( first != NULL ) {
    CircListNode  *second = first->next;
    first-> next = av;    av = second;
    first = NULL;
}
```



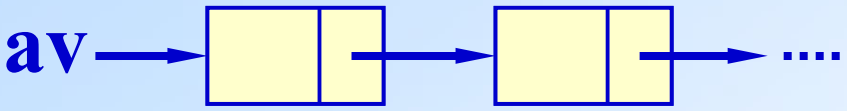
利用可利用空间表回收循环链表（C++描述）

```
if ( first != NULL ) {
    CircListNode<Type> *second = first->link;
    first->link = av;    av = second;
    first = NULL;
}
```

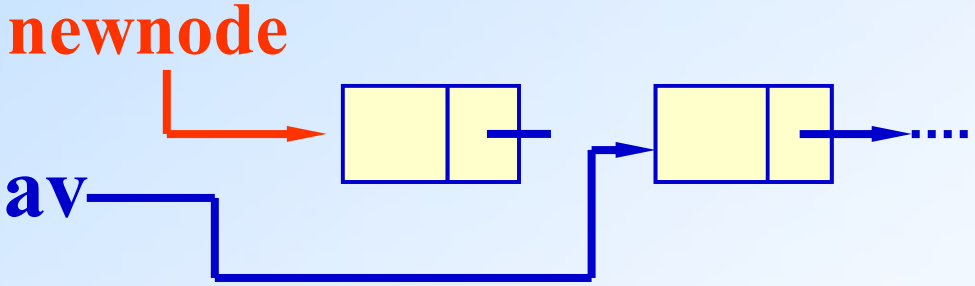


从可利用空间表分配结点（C描述）

```
if ( av == NULL )
        CircListNode  *newnode;
else { newnode = av;  av = av->next; }
```



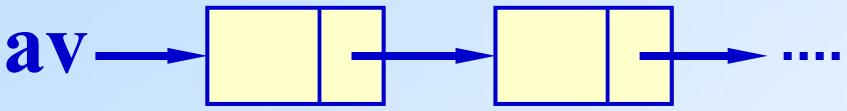
分配前的可利用空间表



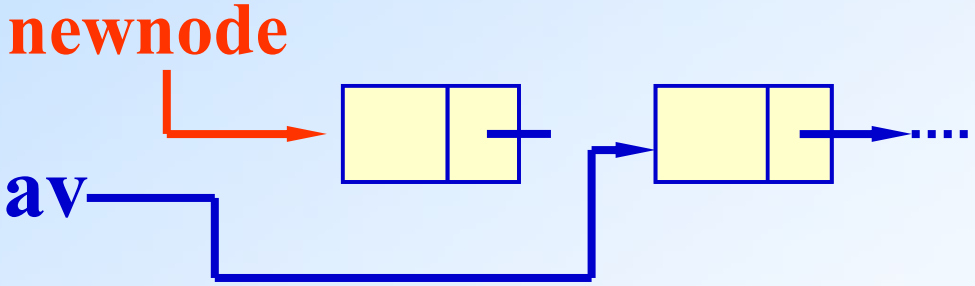
分配后的可利用空间表和新结点

从可利用空间表分配结点（C++描述）

```
if ( av == NULL )
   newnode = new CircListNode<Type>;
else { newnode = av;  av = av->link; }
```



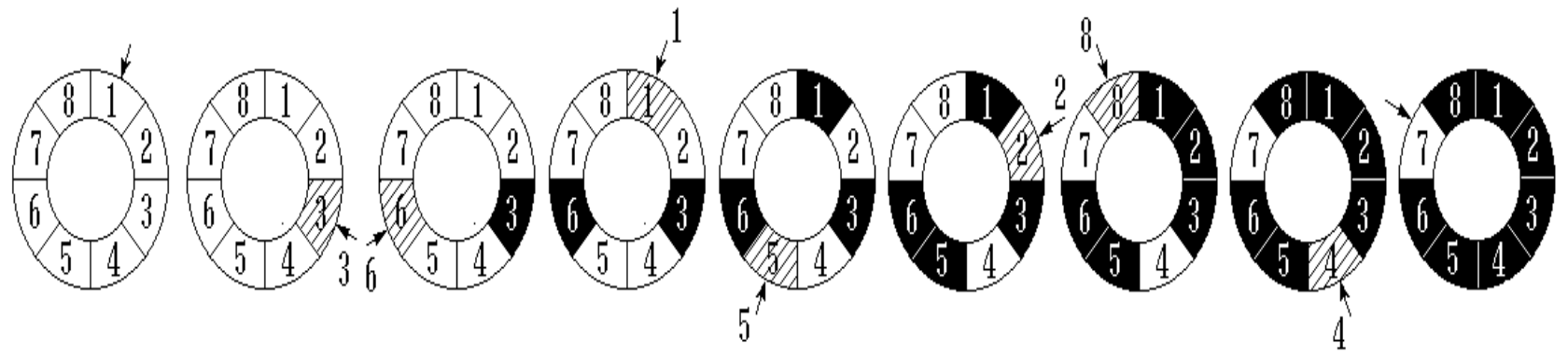
分配前的可利用空间表



分配后的可利用空间表和新结点

约瑟夫问题

- 用循环链表求解约瑟夫问题
- n 个人围成一个圆圈，首先第1个人从1开始一个人一个人顺时针报数，报到第m个人，令其出列。然后再从下一个人开始，从1顺时针报数，报到第m个人，再令其出列，…，如此下去，直到圆圈中只剩一个人为止。此人即为优胜者。
- 例如 n = 8 m = 3



约瑟夫问题的解法

```
#include <iostream.h>
#include “CircList.h”
void Josephus ( int n, int m ) {
    for ( int i=0; i<n−1; i++ ) { //执行n-1次（要删除n−1个数)
        for ( int j=0; j<m−1; j++ ) Next ( );
          //往后数m-1个人
        cout << “Delete person ” <<
              getData ( ) << endl;
        Remove ( );  //删去
    }
}
```

```
void main ( ) {
    CircList<int> clist;
    int n, m;
    cout << “Enter the Number of Contestants?”;
    cin >> n >> m;
    for ( int i=1; i<=n; i++ ) clist.insert (i); //形成约瑟夫环
    clist.Josephus (n, m); //解决约瑟夫问题

}
```

多项式 (Polynomial)

$$P_n(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n$$

$$= \sum_{i=0}^{n} a_i x^i$$

- ***n*阶多项式 *Pn*(*x*) 有 *n*+1 项。**
  - 系数 $a_0, a_1, a_2, \ldots, a_n$
  - 指数 **0, 1, 2, …, *n*，** 按升幂排列

多项式(*Polynomial*)的抽象数据类型（C）

```
typedef struct{
//项的表示，多项式的项作为LinkList的数据元素
        float coef;   //系数
        int expn;     //指数
}term,ElemType;
typedef  LinkList  polynomial;
//用带表头节点的有序链表表示多项式
void CreatPolyn (polynomial &P, int m);
void AddPolyn (polynomial &Pa, polynomial & Pb);
void MultiplyPolyn (polynomial &Pa, polynomial &
Pb);
```

多项式(*Polynomial*)的抽象数据类型（C++）

```
class Polynomial {
public:
    Polynomial ( );                    //构造函数
    int operator ! ( );                //判是否零多项式
    float Coef ( int e);
    int LeadExp ( );                   //返回最大指数
    Polynomial Add (Polynomial poly);
    Polynomial Mult (Polynomial poly);
    float Eval ( float x);             //求值
}
```

创建power类，计算x的幂

```
#include <iostream.h>
class power {
    double x;
    int e;
    double mul;
public:
    power (double val, int exp);          //构造函数
    double get_power ( ) { return mul; }  //取幂值
};
```

```
power :: power (double val, int exp) {
//按val值计算乘幂
    x = val;   e = exp;   mul = 1.0;
    if ( exp == 0 ) return 1;
    for ( ; exp > 0; exp--) mul = mul * x;
}

main ( ) {
    power pwr ( 1.5, 2 );
    cout << pwr.get_power ( ) << "\n";
}
```

创建多项式（C语言）

- void CreatePolyn(polynomail &P, int m){
- //输入m项的系数和指数，建立表示一元多项式的有序链表P
- InitList(P); h = GetHead(P);
- e.coef = 0.0; e.expn = -1; SetCurElem(h,e); //设置头结点的数据元素
- for(i = 1;i<=m; ++1){//依次输入m个非零项

  scanf(e.coef,e.expn);

  if(!LocateElem(P,e,q,(*cmp)())){//当前链表中不存在该指数项

  if(MakeNode(s,e)) InsFirst(q,s); }//生成结点并插入链表
- }
- } //CreatePolyn

```
Status LocateElem (LinkList L, ElemType e, Position &q,
                                        int ( *compare)(ElemType, ElemType));
    // 若有序链表 L 中存在与 e 满足判定函数 compare()取值为 0 的元素，则 q 指示 L 中第一个
    // 值为 e 的结点的位置，并返回 TRUE；否则 q 指示第一个与 e 满足判定函数 compare()取
    // 值>0 的元素的前驱的位置，并返回 FALSE
```

多项式的存储表示（C++）

- 第一种：　**private:**
- (静态数　　**int** degree;
- 组表示)　　**float** coef [maxDegree+1];
- 　　Pn(x)可以表示为：
- 　　pl.degree = n
- 　　pl.coef[i] = a_i,　$0 \le i \le n$



第二种：(动态数组表示)

```
private:
    int degree;
    float * coef;
Polynomial :: Polynomial (int sz) {
    degree = sz;
    coef = new float [degree + 1];
}
```

以上两种存储表示适用于指数连续排列的多项式。但对于指数不全的多项式如 $P_{101}(x) = 3 + 5x^{50} - 14x^{101}$, 不经济。

第三种：（线性表）

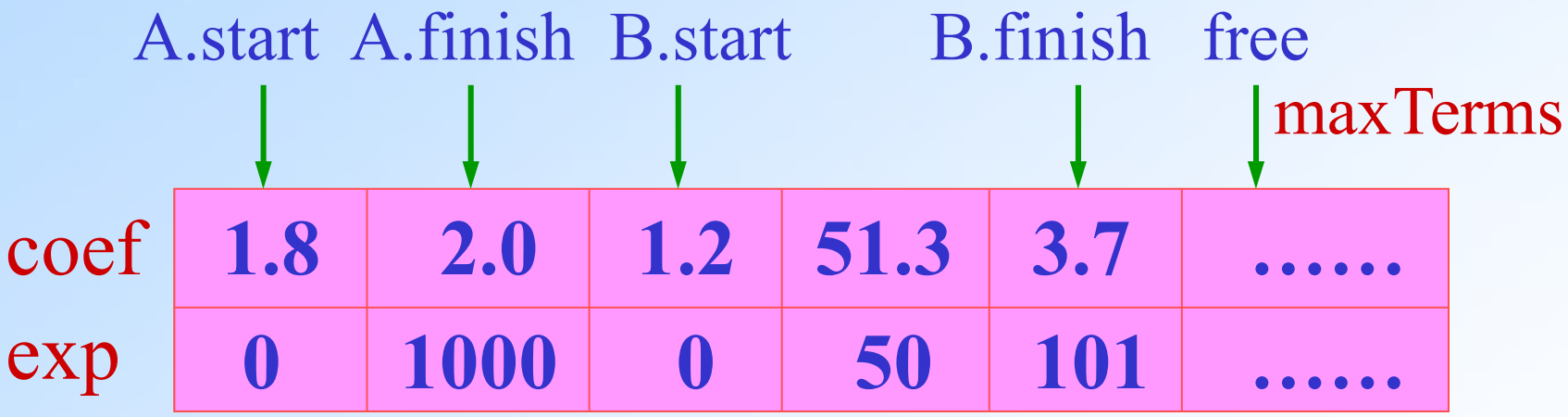
```
class Polynomial;

class term {          //多项式的项定义
friend Polynomial;
private:
   float coef;        //系数
   int exp;           //指数
};
```

	0	1	2		*i*		*m*
coef	a_0	a_1	a_2	……	a_i	……	a_m
exp	e_0	e_1	e_2	……	e_i	……	e_m

$$A(x) = 2.0x^{1000}+1.8$$

$$B(x) = 1.2 + 51.3x^{50} + 3.7x^{101}$$



两个多项式存放在**termArray**中

❖多项式的链表表示及其相加

- 在多项式的链表表示中每个结点增加了一个数据成员link，作为链接指针。

$data \equiv Term$

$coef$	exp	$link$

- 优点是：
  - 多项式的项数可以动态地增长，不存在存储溢出问题。
  - 插入、删除方便，不移动元素。

多项式(polynomial)类的链表定义(C++)

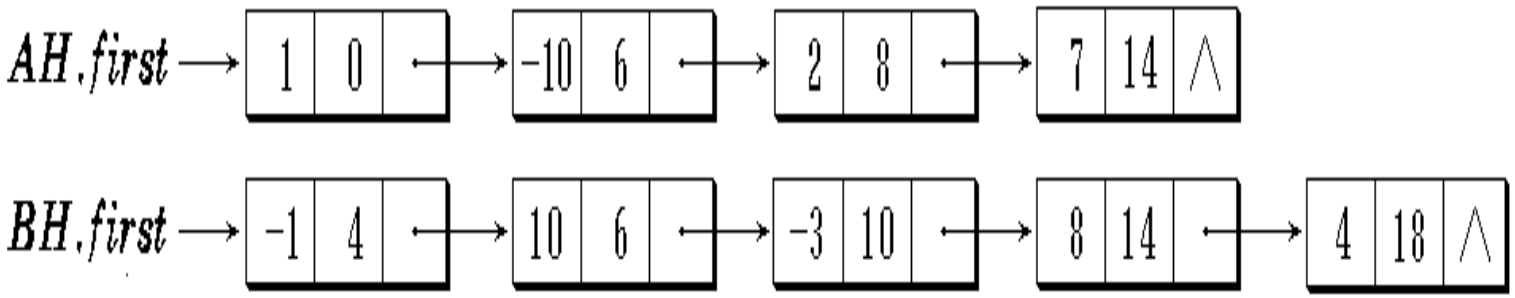
```
struct Term {   //多项式结点定义
    float coef;              //系数
    int exp;                 //指数
    Term * link;
    Term ( float c, int e ) { coef = c;  exp = e; }
};

class Polynomial     {      //多项式类的定义
    List<Term> poly;                  //构造函数
    friend Polynomial & operator +
        ( Polynomial &, Polynomial &); //加法
};
```

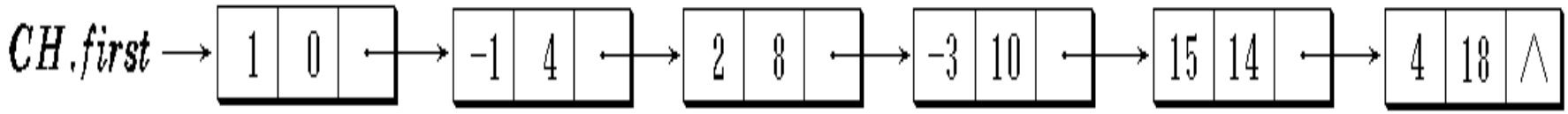
多项式链表的相加

$AH = 1 - 10x^6 + 2x^8 + 7x^{14}$

$BH = -x^4 + 10x^6 - 3x^{10} + 8x^{14} + 4x^{18}$



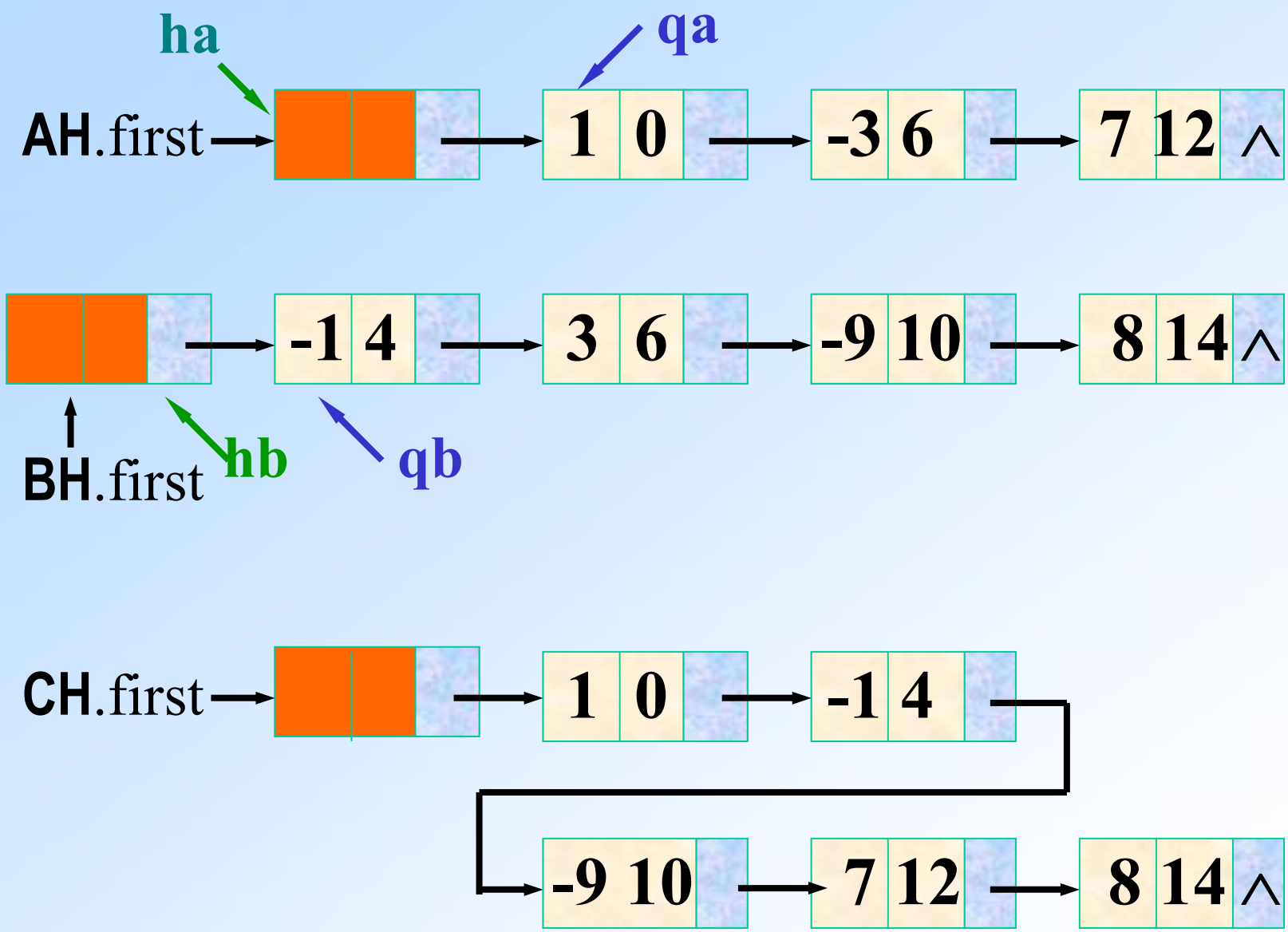
(a) 两个相加的多项式

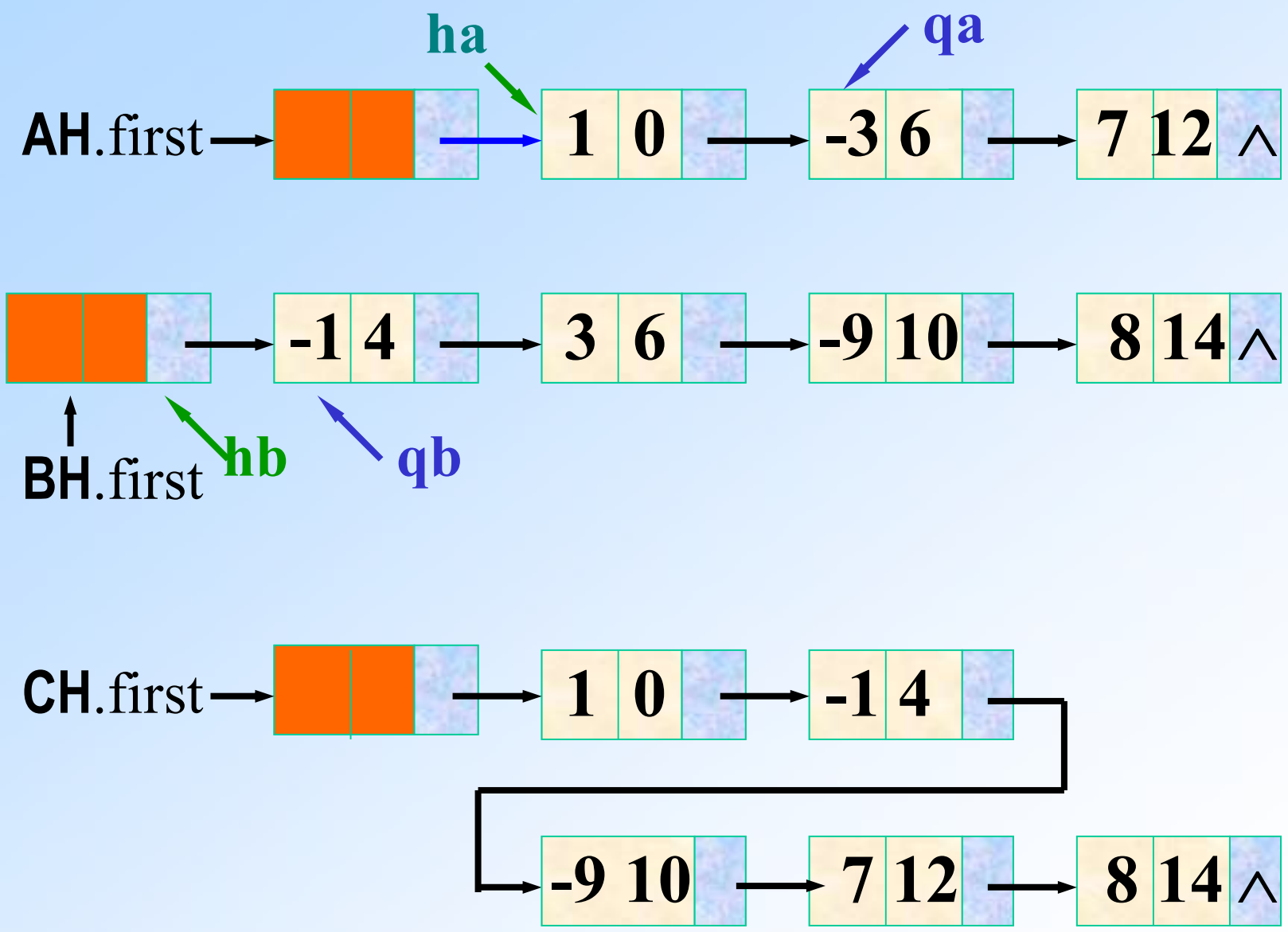


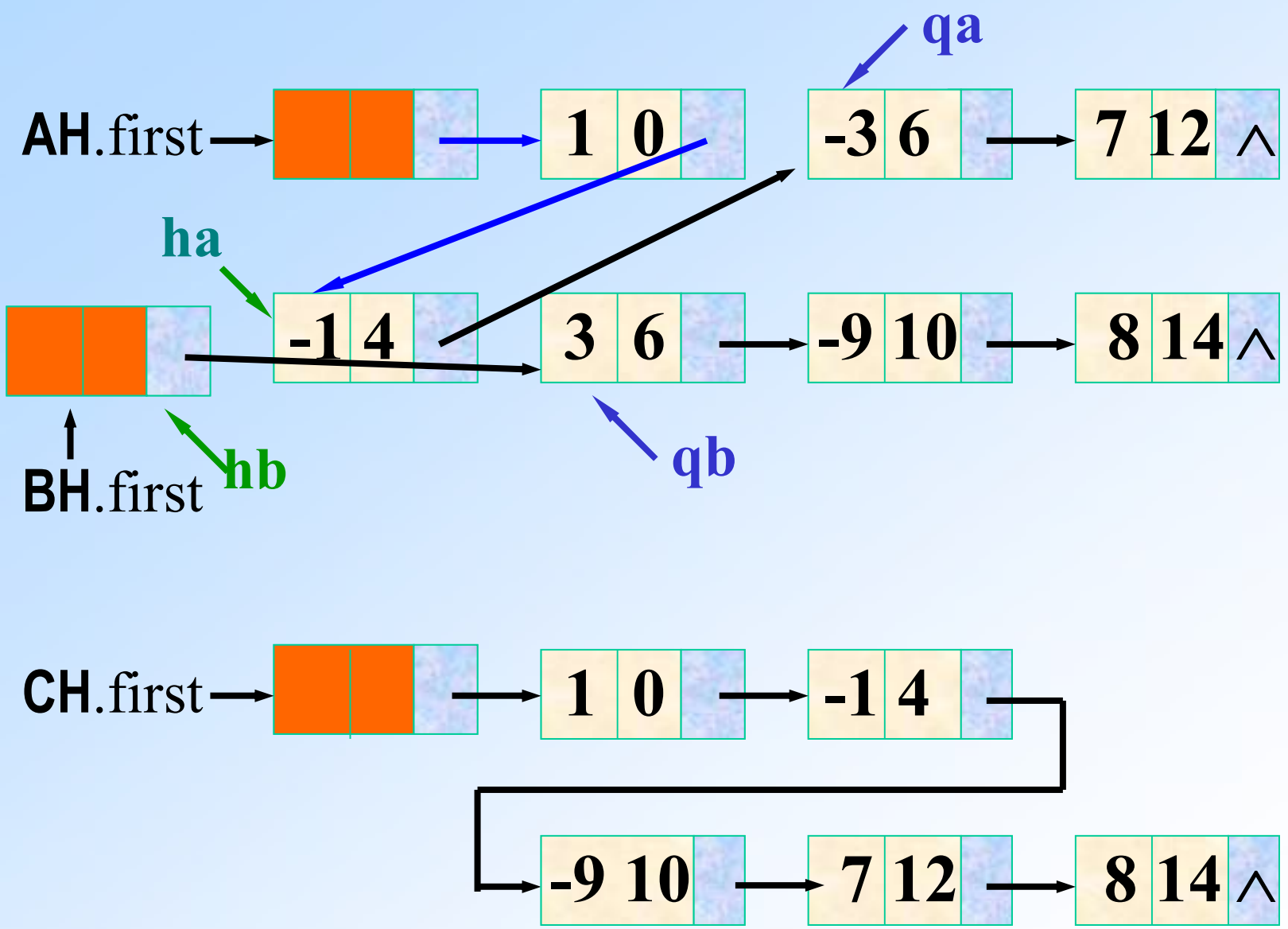
(b) 相加结果的多项式

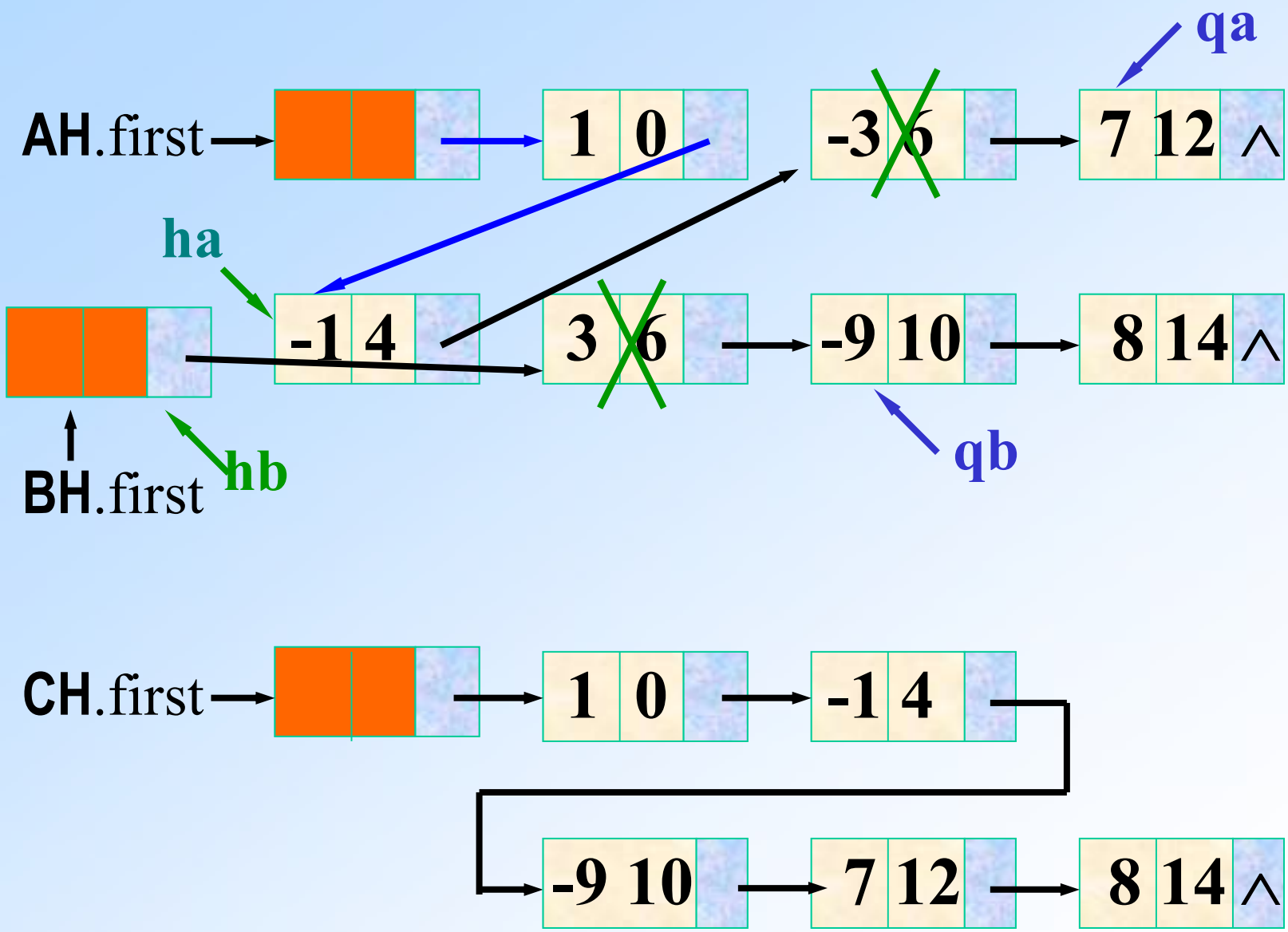
两个多项式的相加算法

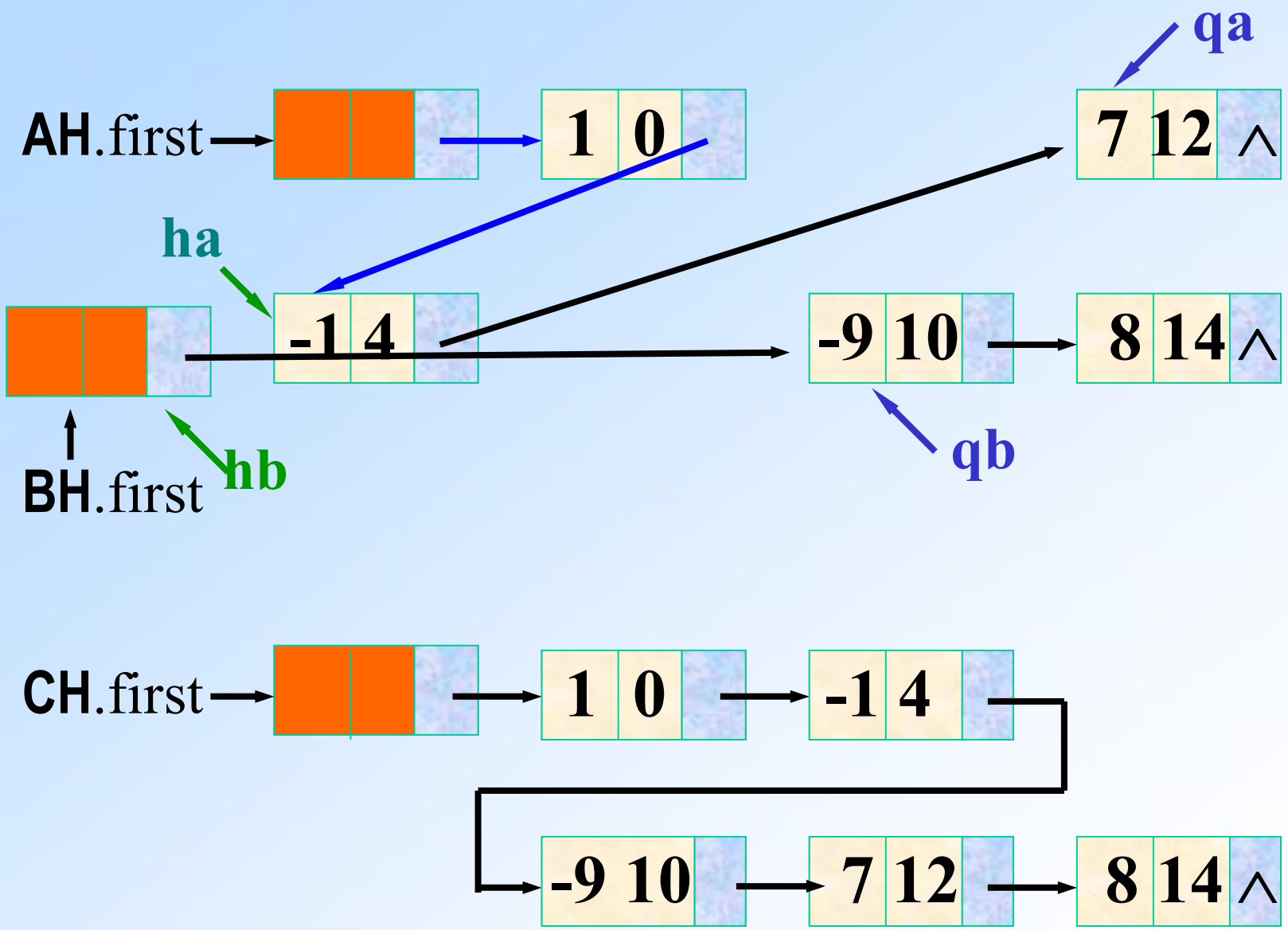
- 扫描两个多项式，若都未检测完：
  - 若当前被检测项指数相等，系数相加。若未变成 **0**，则将结果加到结果多项式。
  - 若当前被检测项指数不等，将指数小者加到结果多项式。
- 若一个多项式已检测完，将另一个多项式剩余部分复制到结果多项式。

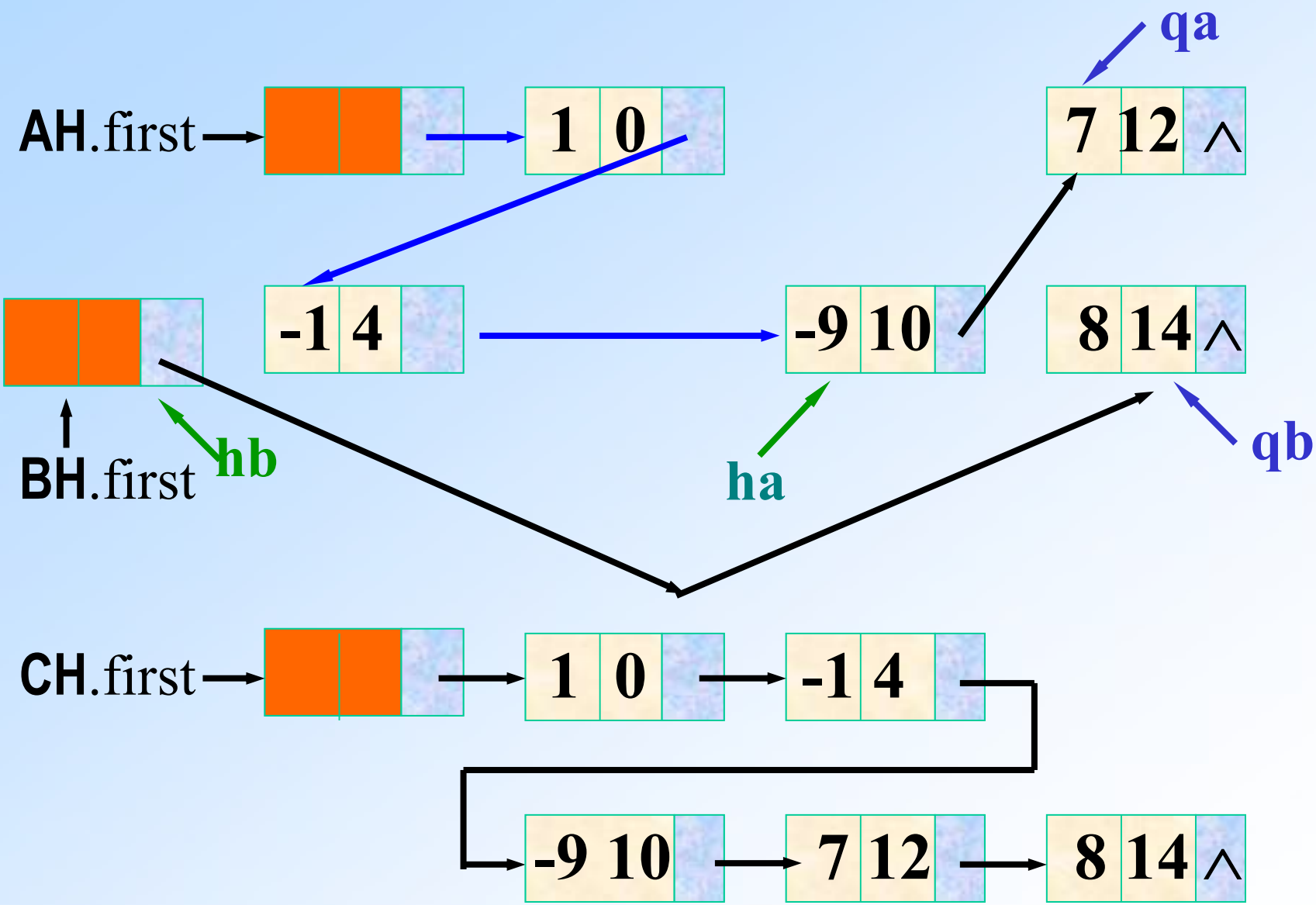

ha
qa
AH.first
1 0
-3 6
7 12 ∧
-1 4
3 6
-9 10
8 14 ∧
BH.first
hb
qb
CH.first
1 0
-1 4
-9 10
7 12
8 14 ∧

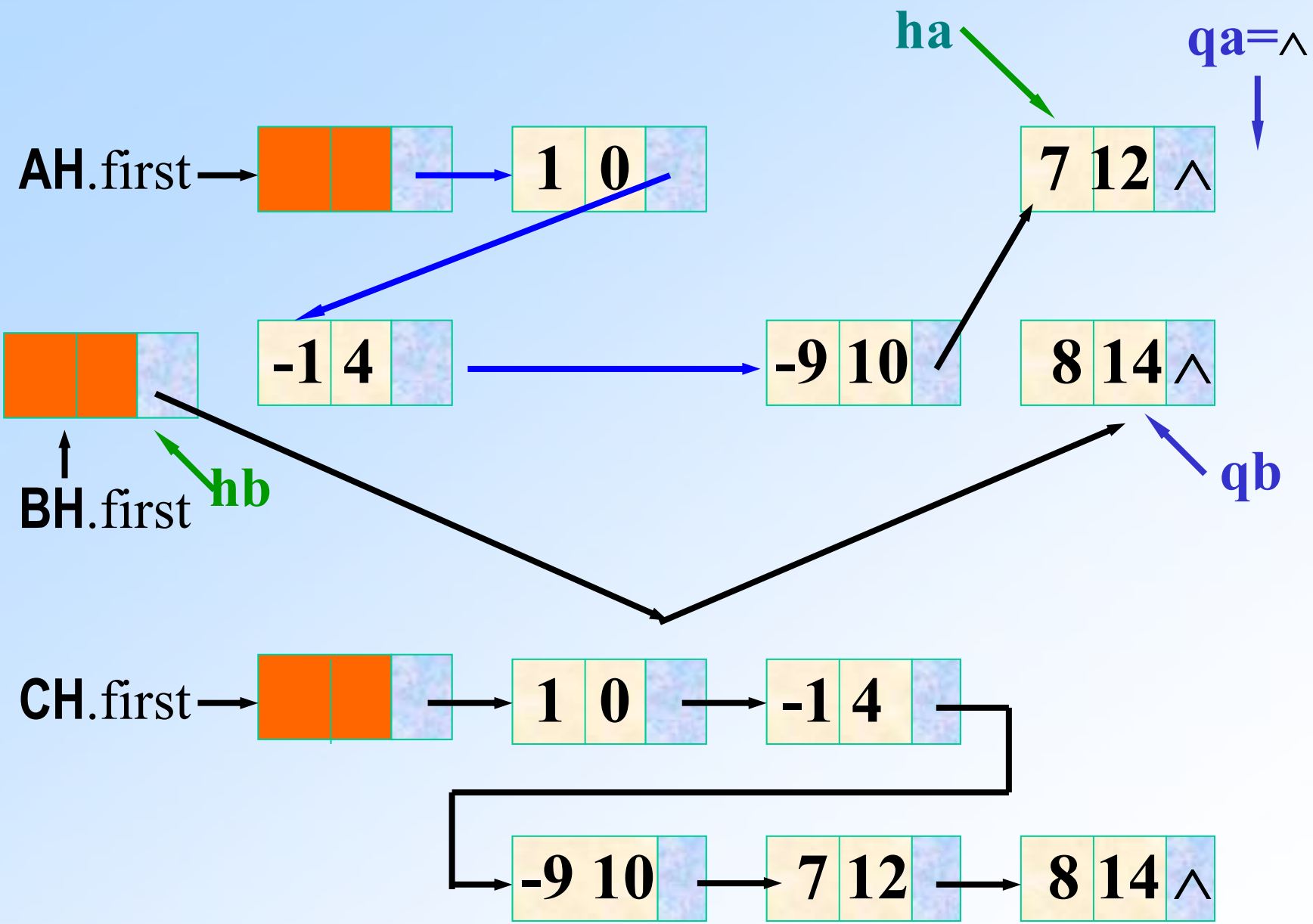

ha
qa
AH.first
1 0
-3 6
7 12 ∧
-1 4
3 6
-9 10
8 14 ∧
BH.first
hb
qb
CH.first
1 0
-1 4
-9 10
7 12
8 14 ∧

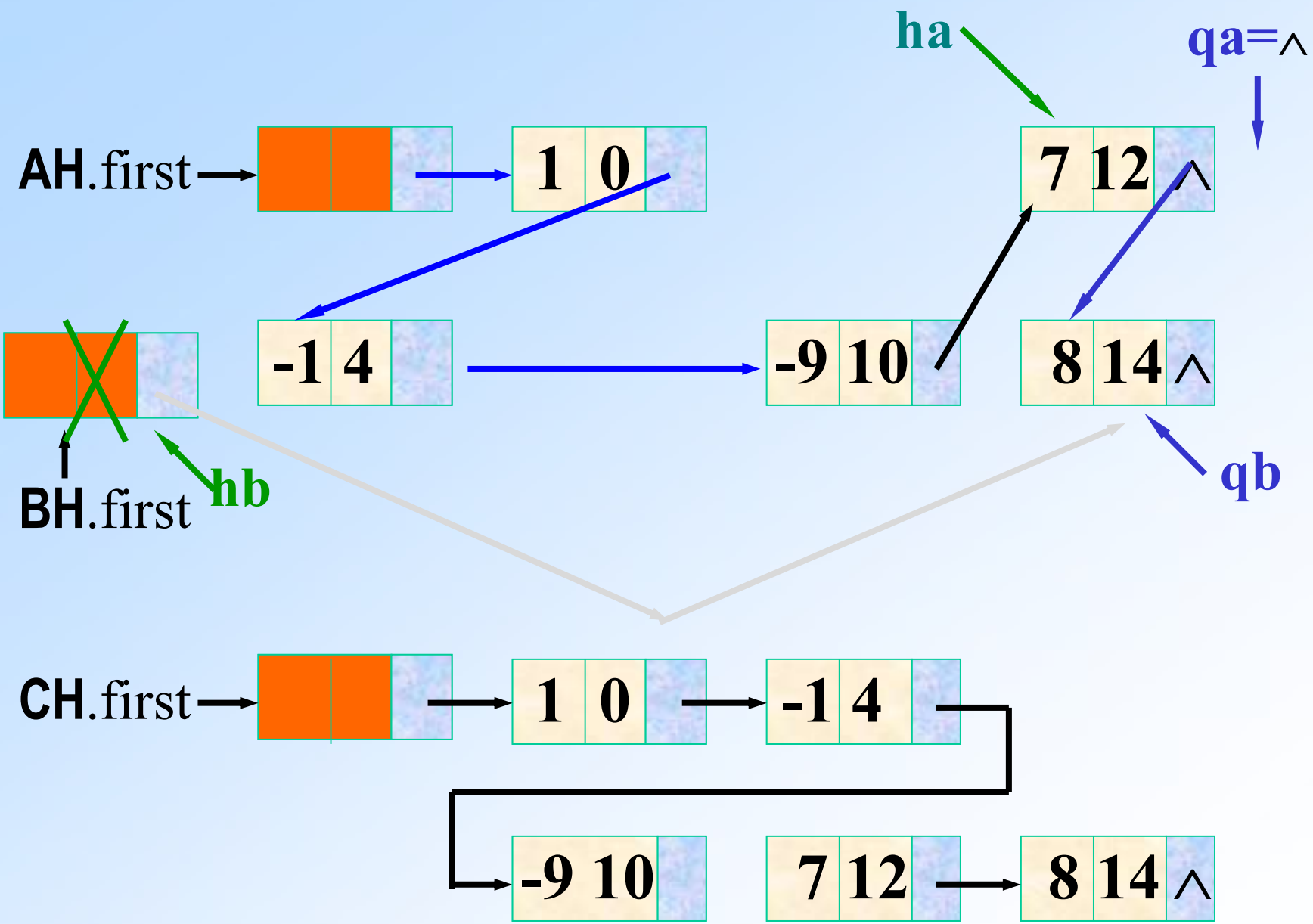

qa
AH.first
1 0
-3 6
7 12 ∧
ha
-1 4
3 6
-9 10
8 14 ∧
BH.first
hb
qb
CH.first
1 0
-1 4
-9 10
7 12
8 14 ∧


AH.first
1 0
-3 6
7 12 ∧
qa
ha
-1 4
3 6
-9 10
8 14 ∧
qb
BH.first
hb
CH.first
1 0
-1 4
-9 10
7 12
8 14 ∧


AH.first
1 0
7 12 ∧
qa
ha
-1 4
-9 10
8 14 ∧
qb
BH.first
hb
CH.first
1 0
-1 4
-9 10
7 12
8 14 ∧


AH.first
1 0
7 12 ∧
qa
-1 4
-9 10
8 14 ∧
BH.first
hb
ha
qb
CH.first
1 0
-1 4
-9 10
7 12
8 14 ∧


ha
qa=∧
AH.first
1 0
7 12 ∧
-1 4
-9 10
8 14 ∧
BH.first
hb
qb
CH.first
1 0
-1 4
-9 10
7 12
8 14 ∧


ha
qa=^
AH.first
1 0
7 12 ^
-1 4
-9 10
8 14 ^
BH.first
hb
qb
CH.first
1 0
-1 4
-9 10
7 12
8 14 ^

C算法描述

```
void AddPolyn ( Polynomial& Pa, Polynomial& Pb )
{   //多项式Pa= Pa+Pb,
    ha = GetHead(Pa); hb = GetHead(Pb);
    //ha和hb分别指向Pa和Pb的头结点
    qa = NextPos(Pa,ha); qb = NextPos(Pb,hb);
    //qa和qb分别指向Pa和Pb的当前结点
       while(qa && qb) {   //qa和qb均非空
        a=GetCurElem(qa); b=GetCurElem(qb);
        //a和b为两表中当前比较元素
        switch(*cmp(a,b)){
             case -1: //多项式Pa中当前结点的指数值小
                  ha = qa; qa = NextPos(Pa,qa); break;
```

```
        case 0 :                //两者的指数相等
          sum = a.coef + b.coef;
          if(sum!=0.0){
        //修改多项式Pa中当前结点的系数值
          SetCurElem(qa.sum); ha = qa;  }
          else {                //删除多项式PA中当前结点
            DelFirst(ha,qa); FreeNode(qa);  }
          DelFirst(hb,qb); FreeNode(qb);
          qb=NextPos(Pb,hb);
          qa=NextPos(Pa,ha);break;
        case1 : //多项式PB中当前结点的指数值小
            DelFirst(hb,qb);  InsFirst(ha,qb);
            qb = NextPos(Pb,hb);
}//switch
```

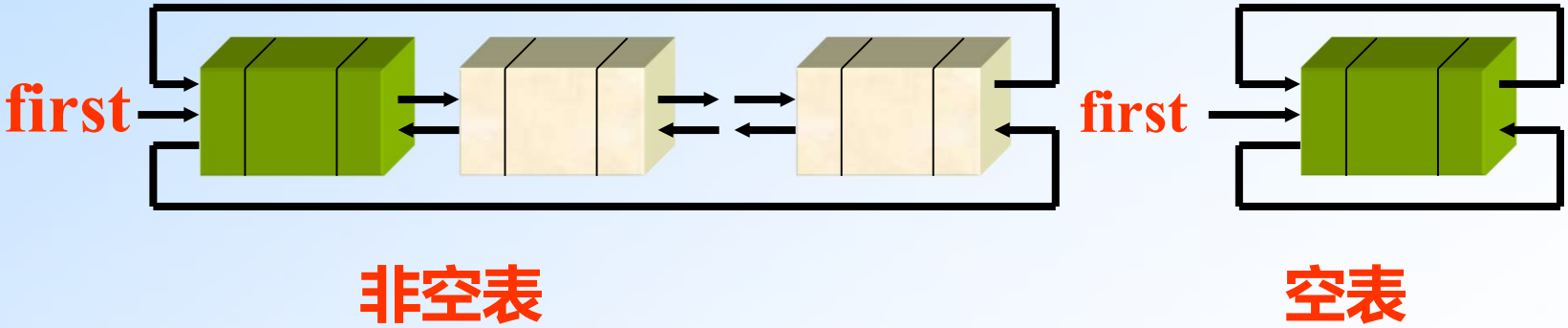
```
}//while
if ( !ListEmpty(Pb))    Append(Pa,qb);
          //链接Pb中剩余结点
    FreeNode(hb); //释放Pb的头结点
} //AddPolyn
```



双向链表 (Doubly Linked List)

- 双向链表结点结构：





双向循环链表的定义

```
typedef int ListData;
typedef struct dnode {
    ListNode data;
    struct dnode * prior, * next;
} DblNode;
typedef DblNode * DblList;
```

建立空的双向循环链表

```
void CreateDblList ( DblList & first ) {
    first = ( DblNode * ) malloc
                ( sizeof ( DblNode ) );
    if ( first == NULL )
        { print ( "存储分配错!\n" );  exit (1); }
    first->prior = first->next = first;
          //表头结点的链指针指向自己
}
```

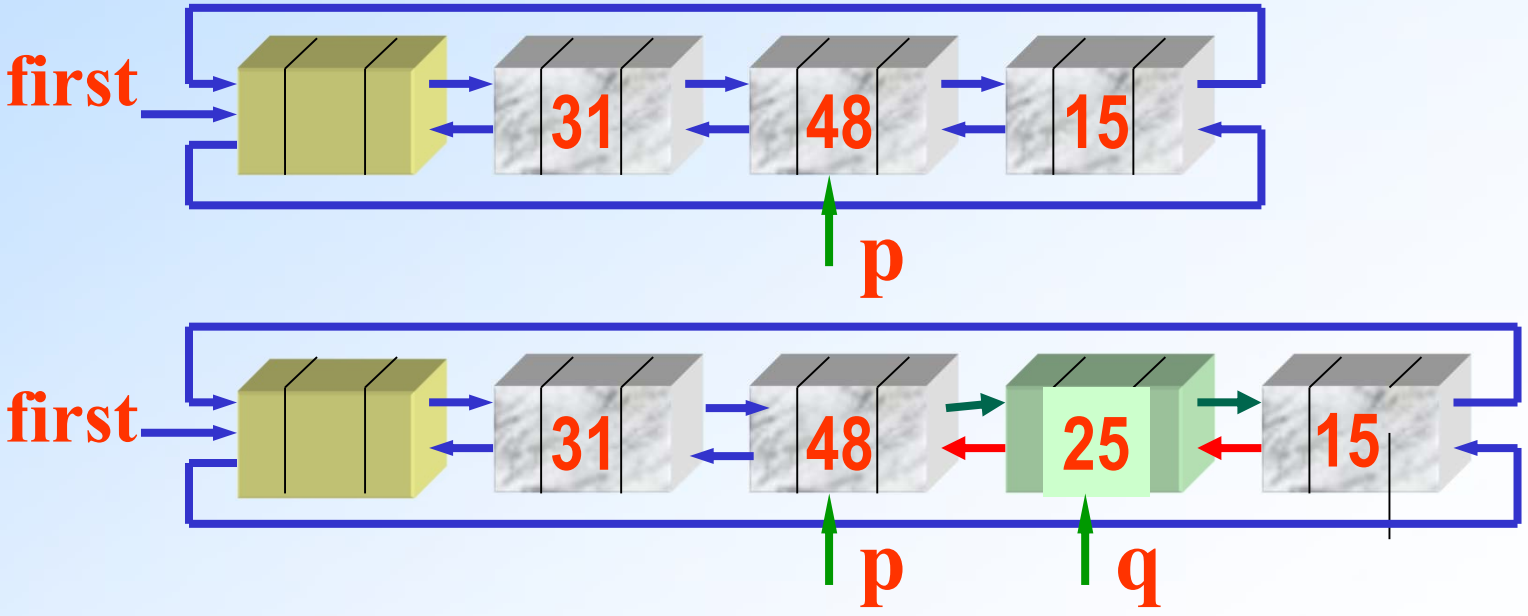
计算双向循环链表的长度

```
int Length ( DblList first ) {
//计算带表头结点的双向循环链表的长度
    DblNode * p = first->next;
    int count = 0;
    while ( p != first )
        { p = p->next;  count++; }
    return count;
}
```

双向循环链表的插入 (非空表)

在结点 *p 后插入25

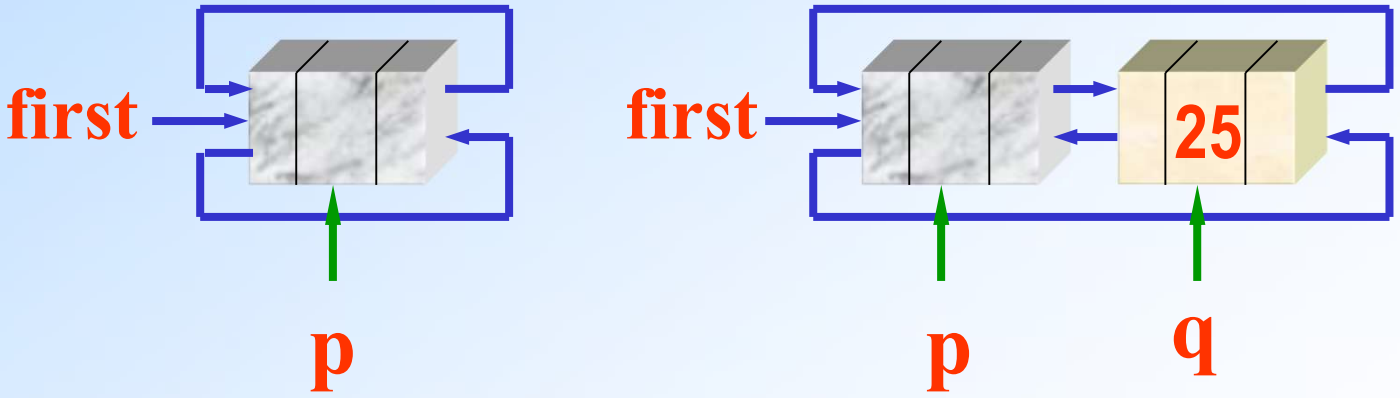
```
q->prior = p;
q->next = p->next;
p->next = q;
q->next->prior = q;
```



双向循环链表的插入 (空表)

在结点 *p 后插入25

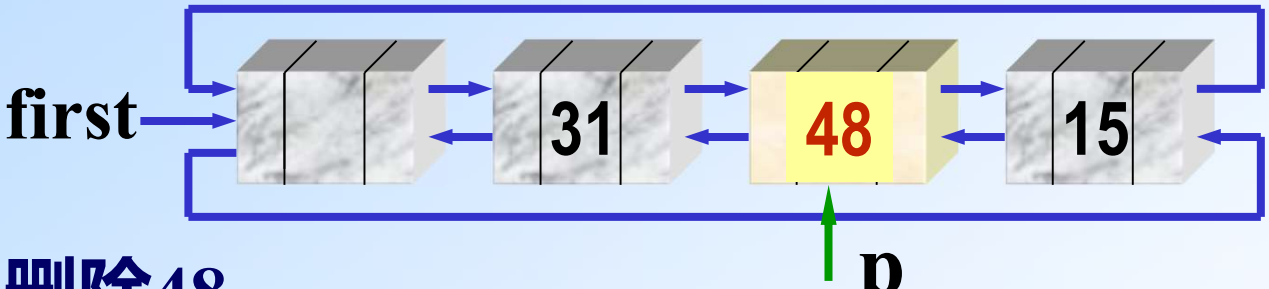
```
q->prior = p;
q->next = p->next;
 p->next = q;
q->next->prior = q;
```



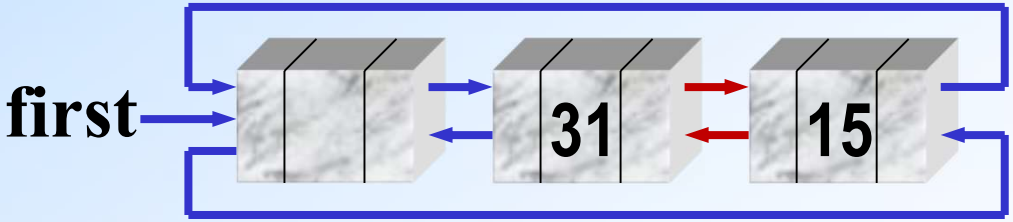
```
int Insert ( DblList first,  int i, ListData x ) {
    DblNode * p = Locate ( first, i-1 );
    //指针定位于插入位置
    if ( p == first && i != 1) return 0;
    DblNode * q = ( DblNode * ) malloc
          ( sizeof ( DblNode ) );     //分配结点
    q->data = x;
    q->prior = p;    p->next->prior = q;
    //在前驱方向链入新结点
    q->next = p->next;  p->next = q;
    //在后继方向链入新结点
    return 1;
}
```

双向循环链表的删除

```
p->next->prior = p->prior;
p->prior->next = p->next;
```

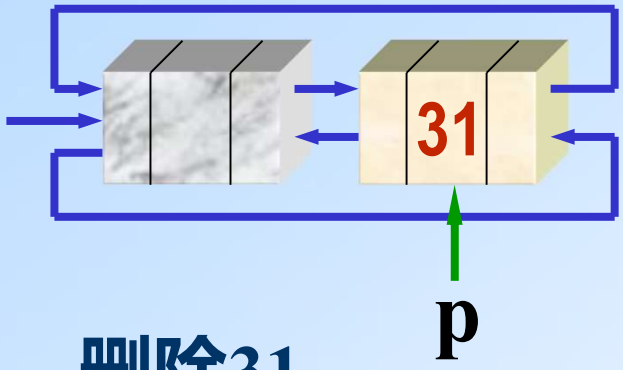


删除48

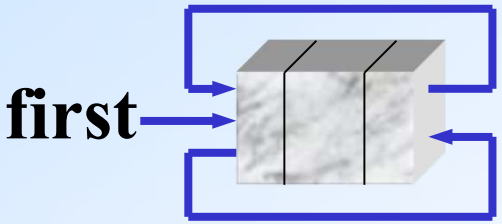


(非空表)

双向循环链表的删除



删除31




```
p->next->prior = p->prior;
p->prior->next = p->next;
```

```
int Remove ( DblList first, int i ) {
    DblNode * p = Locate ( first, i );
    //指针定位于删除结点位置
    if ( p == first ) return 0;
    p->next->prior = p->prior;
    p->prior->next = p->next;
    //删除结点 p
    free ( p );        //释放
    return 1;
}
```

顺序表与链表的比较

基于空间的比较

- 存储分配的方式
  - 顺序表的存储空间是静态分配的
  - 链表的存储空间是动态分配的
- 存储密度 = 结点数据本身所占的存储量/结点结构所占的存储总量
  - 顺序表的存储密度 = 1
  - 链表的存储密度 < 1

顺序表与链表的比较

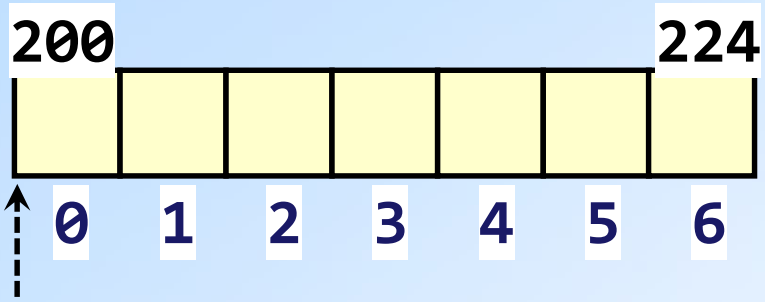
基于时间的比较

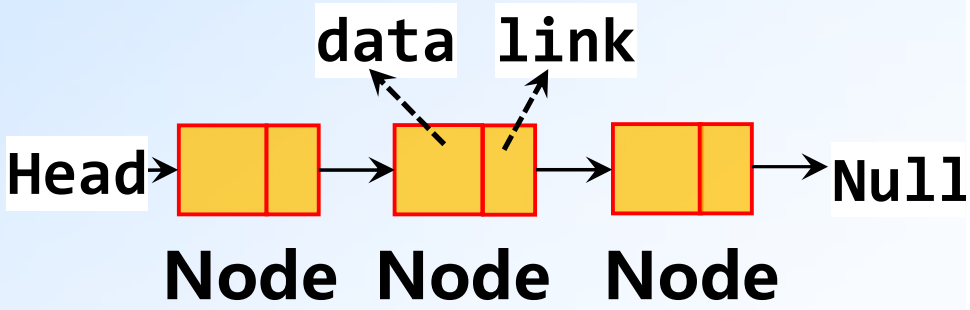
- 存取方式
  - 顺序表可以随机存取，也可以顺序存取
  - 链表是顺序存取的
- 插入/删除时移动元素个数
  - 顺序表平均需要移动近一半元素
  - 链表不需要移动元素，只需要修改指针

顺序表与链表比较

顺序表	链表
全局坐标地址访问	局部邻域关系访问

- 元素访问时间





A[i]的地址:

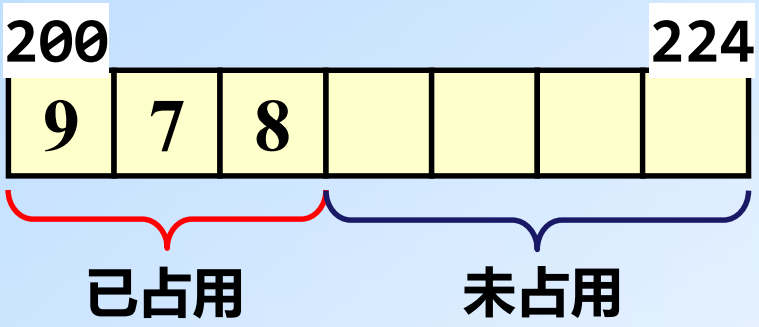
时间复杂度：O(1)

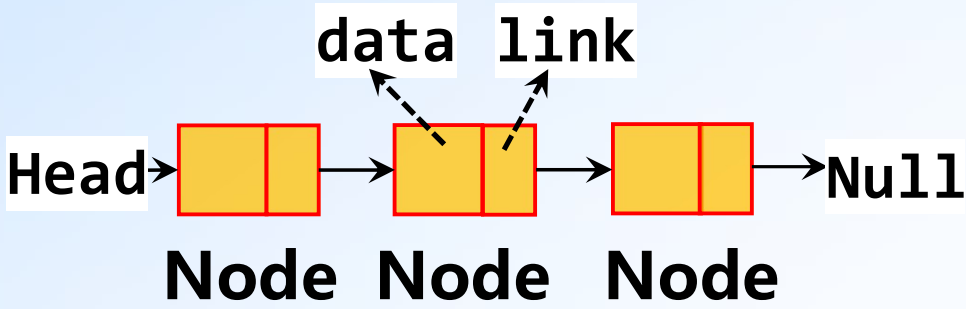
平均时间复杂度：O(n)

顺序表与链表比较

顺序表	链表
全局坐标地址访问	局部邻域关系访问

- 内存占用





- ✓ 没有空间浪费
- ✓ 需要指针

若data为4字节

4×7=28字节

(4+4)× 3=24字节

若data为16字节

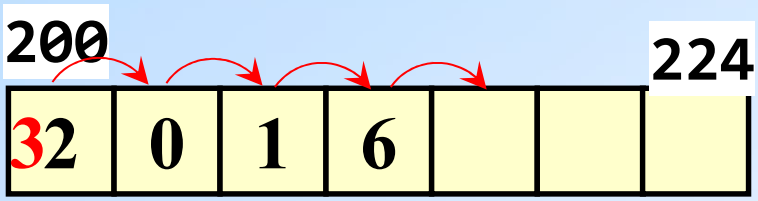
16×7=112字节

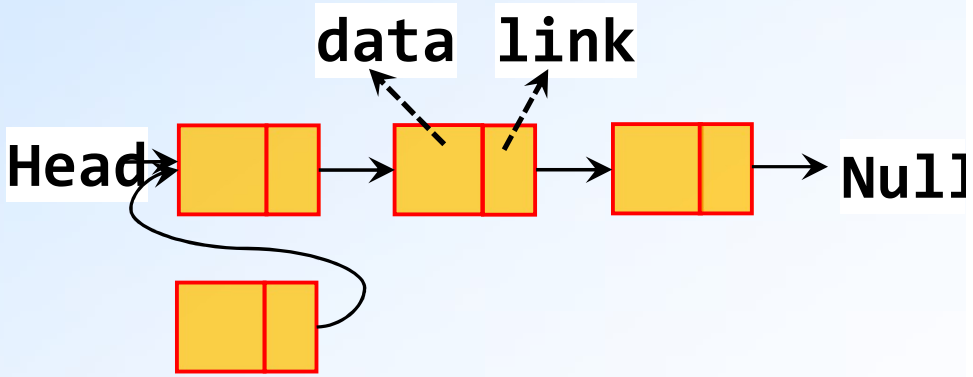
(16+4)×3=60字节

顺序表与链表比较

顺序表	链表
全局坐标地址访问	局部邻域关系访问

- 元素插入时间





在开头插入	O(n)	O(1)
在尾部插入	O(1), 向量未满时	O(n)
在中部插入	O(n)	O(n)

插入后的移动时间　　插入前的访问时间

结　束

下面是采用C++描述的顺序表的定义及操作，可供参考

顺序表(SeqList)类的定义

```
template <class Type>   class SeqList {
      Type *data;        //顺序表存储数组
      int MaxSize;     //最大允许长度
      int last;          //当前最后元素下标
public:
      SeqList ( int MaxSize = defaultSize );
      ~SeqList ( ) { delete [ ] data; }
      int Length ( ) const { return last+1; }
```

C++描述

```
int Find ( Type& x ) const;          //查找
    int Locate ( int i ) const;          //定位
    int Insert ( Type & x, int i );      //插入
    int Remove ( Type & x );             //删除
    int Next ( Type & x ) ;              //后继
    int Prior ( Type & x ) ;             //前驱
    int IsEmpty ( ) { return last ==-1; }
    int IsFull ( ) { return last == MaxSize-1; }
    Type Get ( int i ) {                 //提取
        return i < 0 || i > last?  NULL : data[i];
    }
}
```

顺序表部分公共操作的实现

```
template <class Type>                    //构造函数
SeqList<Type> :: SeqList ( int sz ) {
   if ( sz > 0 ) {
        MaxSize = sz;  last = -1;
        data = new Type[MaxSize];
       if ( data == NULL ) {
           MaxSize = 0;   last = -1;
           return;
       }
    }
}
```

C++描述

```
template <class Type>
int SeqList<Type> :: Find ( Type & x ) const {
//搜索函数：在顺序表中从头查找结点值等于
//给定值x的结点
   int  i = 0;
      while ( i <= last && data[i] != x )
         i++;
      if ( i > last ) return -1;
      else return i;
}
```

C++描述

```
template <class Type>
int SeqList<Type> :: Insert (Type& x, int i ) {
//在表中第 i 个位置插入新元素 x
    if ( i < 0 || i > last+1 || last == MaxSize-1 )
      return 0;                    //插入不成功
    else {
      last++;

      for ( int j = last; j > i; j-- )
                        data[j] = data[j -1];
      data[i] = x;   return 1;    //插入成功
    }
}
```

C++描述

```
template <class Type>
int SeqList<Type> :: Remove ( Type& x ) {
//在表中删除已有元素 x
    int i = Find (x);              //在表中搜索 x
    if ( i >= 0 ) {
        last-- ;
     for ( int j = i; j <= last; j++ )
            data[j] = data[j+1];
        return 1;                  //成功删除
    }
    return 0;                      //表中没有 x
}
```

顺序表的应用：集合的“并”运算

```
void Union ( SeqList<int> & A,
                          SeqList<int> & B ) {
    int n = A.Length ( );
    int m = B.Length ( );
    for ( int i = 0; i < m; i++ ) {
      int x = B.Get(i);          //在B中取一元素
      int k = A.Find (x);        //在A中搜索它
      if ( k == -1 )             //若未找到插入它
          { A.Insert (n, x);  n++; }
    }
}
```

顺序表的应用：集合的“交”运算

```
void Intersection ( SeqList<int> & A,
                    SeqList<int> & B ) {
    int n = A.Length ( );
    int m = B.Length ( );  int i = 0;
    while ( i < n ) {
       int x = A.Get (i);      //在A中取一元素
      int k = B.Find (x);    //在B中搜索它
      if ( k == -1 ) { A.Remove (i);  n--; }
      else i++;                 //未找到在A中删除它
    }
}
```

单链表的类定义(C++描述)

- 多个类表达一个概念(单链表)。
  - 链表结点(ListNode)类
  - 链表(List)类
  - 链表游标(Iterator)类
- 定义方式
  - 复合方式
  - 嵌套方式
  - 继承方式

```
class List;                 //链表类定义（复合方式）

class ListNode {            //链表结点类
friend class List;          //链表类为其友元类
private:
   int data;                //结点数据, 整型
   ListNode * link;         //结点指针
};

class List {                //链表类
private:
   ListNode *first , *last;     //表头指针
};
```

```
class List {          //链表类定义(嵌套方式)
private:
    class ListNode {          //嵌套链表结点类
    public:
        int data;
        ListNode *link;
    };
    ListNode *first , *last;    //表头指针
public:
    //链表操作.........
};
```

链表类和链表结点类定义(继承方式)

```
class ListNode {          //链表结点类
protected:
    int data;
    ListNode * link;
};

class List : public class ListNode {
//链表类,  继承链表结点类的数据和操作
 private:
     ListNode *first , *last;       //表头指针
};
```

循环链表类的定义 (C++表示)

```
template <class Type> class CircList;

template <class Type> class CircListNode {
friend class CircList <Type>;
public:
    CircListNode ( Type d = 0,
        CircListNode<Type> *next = NULL )
        : data ( d ), link ( next ) { }    //构造函数
private:
    Type data;                             //结点数据
    CircListNode<Type> *link;              //链接指针
}
```

```
template <class Type> class CircList {
private:
    CircListNode<Type> *first, *current, *last;
    //链表的表头指针、当前指针和表尾指针
public:
    CircList ( Type value );
    ~CircList ( );
    int Length ( ) const;
    Boolean IsEmpty ( )
        { return first->link == first; }
    Boolean Find ( const Type & value );
```

```
    Type getData ( ) const;
    void Firster ( ) { current = first; }
    Boolean First ( );
    Boolean Next ( );
    Boolean Prior ( );
    void Insert ( const Type & value );
    void Remove ( );
};
```

多项式相加的C++算法描述

```
Polynomial& operator +
    ( Polynomial& ah, Polynomial& bh ) {
//两个带头结点的按升幂排列的多项式相加，
//返回结果多项式链表的表头指针，结果不
//另外占用存储，覆盖ah和bh链表
   ListNode<Term> *pa, *pb, *pc, *p;
   Term a, b;
   pa=pc = ah.poly.First ( ); //结果存放指针
   pb=p = bh.poly.First ( );
   pa = ah.poly.Next( );     //多项式检测指针
   pb = bh.poly.Next( );     //多项式检测指针
   delete p;                 //删去bh的表头结点
```

```
while ( pa != NULL && pb != NULL ) {
   a = pa->data;  b = pb->data;
   switch ( compare ( a.exp, b.exp ) ) {
   case '==' :              //pa->exp == pb->exp
        a.coef = a.coef + b.coef;   //系数相加
        p = pb;  pb = pb->link;
        delete p;           //删去原pb所指结点
        if ( abs(a.coef ) < 0.0001) {
           p = pa;  pa = pa->link;  delete p;
        }                   //相加为零, 该项不要
        else {              //相加不为零, 加入ch链
           pa->data = a;   pc->link = pa;
```

```
            pc = pa;  pa = pa->link;
        }
        break;
    case '>' :              //pa->exp > pb->exp
        pc->link = pb;  pc = pb;
        pb = pb->link;
        break;
    case '<' :              //pa->exp < pb->exp
        pc->link;
        pc = pa;  pa = pa->link;
    }
}
```

```
    if ( pa->link ) pc->link = pa;
    else pc->link = pb;     //剩余部分链入ch链
    return ah;
}
```

